Tarea 2 - Cerradura Convexa

Equipo Docente: Arianne Peña, Gustavo Medel, Nancy Hitschfeld K.

1. Introducción

La cerradura convexa (ver Figura 1), aparte de ser un conjunto matemático de puntos que cumplen ciertas condiciones, es algo que va más allá de la teoría y se utiliza activamente en distintas aplicaciones. Tales como reconocimiento de patrones, procesamiento de imágenes, información geográfica e incluso cosas como la teoría de juegos.

Por todo ello y mucho más, hoy en día existen al menos 8 formas distintas de implementar dicho algoritmo y el objetivo principal de esta tarea, será probar al menos dos de ellas utilizando como base su Tarea 1, realizando el trabajo en parejas de así desearlo.

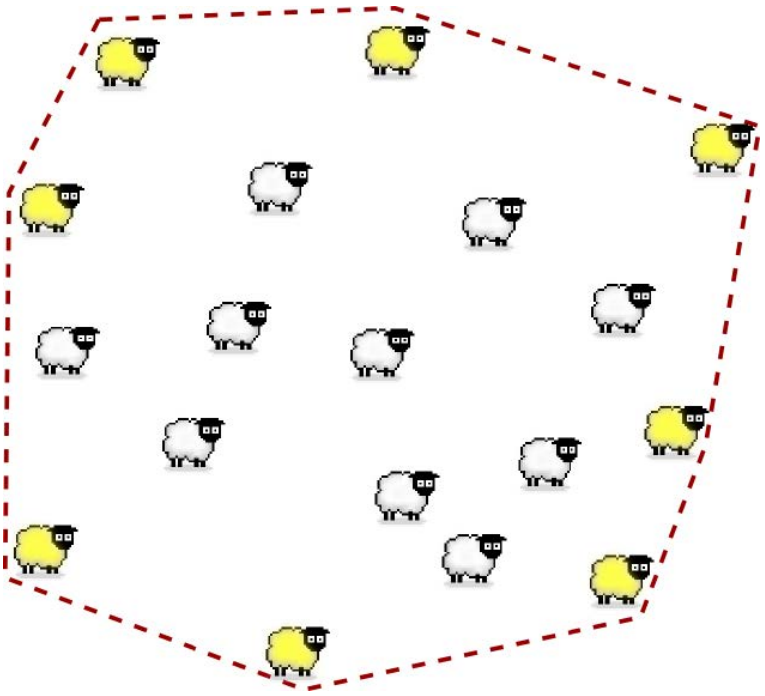


Figura 1: Ejemplo de una cerradura convexa compuesta por 8 lindas ovejitas.

2. Elementos a implementar

Esta tarea será implementada sobre su Tarea 1 y, por tanto, **toda debe estar desarrollada en C++**. Se recomienda crear una nueva clase de Estrategias de Cerradura Convexa (Patrón de diseño Strategy) de la cual heredan la clase IncrementalAlgorithm y el otro algoritmo a implementar. Debe existir un metodo *apply*, que se sobreescribe e implementa la estrategia de esa clase.

La tarea está dividida en tres secciones:

1. Debes implementar el algoritmo **Incremental**, el cual debe recibir como argumento un listado (o nube) de puntos y debe retornar el polígono que representa la cerradura. Puede usted guiarse por la siguiente firma: `Polygon apply(Point[] cloud)`.

2. El algoritmo incremental es $O(nlogn)$, recordemos que su funcionamiento es ordenar los puntos, construir el upper-hull y luego devolverse y construir el lower-hull.
   Como podemos estar trabajando con nubes de millones de puntos, es necesario poder utilizar otro algoritmo similar para poder comparar. Por ello, debes implementar el algoritmo **Random Incremental** (que siga la misma firma que Incremental).
   Este algoritmo es similar al incremental observado en clase, pero su particularidad es que comienza de una cápsula convexa simple de pocos puntos (por ejemplo, 3, donde un triángulo es la única posible) y luego inserta puntos modificando la cápsula convexa del paso anterior y así sucesivamente. Una definición de este algoritmo se encuentra en la sección 2 del siguiente artículo: https://www.cs.cmu.edu/~15451-f20/LectureNotes/2D-ConvexHull.pdf.

3. Una vez implementado ambos métodos, diseña un experimento (ver lectura 2) para determinar para qué distribuciones de puntos es mejor usar uno u otro. Por ejemplo, ¿Cuál es mejor con puntos generados aleatoriamente? ¿Cuál es mejor si un porcentaje pequeño de puntos pertenece a la cerradura convexa? ¿Se puede determinar ese porcentaje?

   Implementa además un método que genere una lista aleatoria de n puntos, y otro que genere un conjunto de puntos en que un cierto porcentaje de estos forme parte de la cerradura convexa y el resto se distribuya aleatoriamente en el interior. Estos métodos construyen la lista de puntos con que se llamarán a continuación los algoritmos de cerradura convexa. Estos algoritmos tambien se pueden implementar usando el patrón de diseño Strategy asi es fácil agregar nuevas estrategias de generación de conjuntos de puntos de prueba. Experimente con distintos tamaños de n, por ejemplo $n = 10^4; 10^5; 10^6$ y con distinto porcentaje de puntos formando parte de la cerradura. ¿Cómo se desempeñan los algoritmos? Existen diferencias en cuanto a tiempo de cálculo? Qué diferencias existen si los conjuntos de puntos son reales o si son enteros? Pruébalo en particular, para un conjuntos de puntos en que la cerradura convexa tiene puntos colineales y otro que no tenga puntos colineales. No es necesario que tus algoritmos funcionen de manera consistente en ambos casos pero si que lo comentes en el informe.

OJO: También debe comparar que una vez terminado el procesamiento de ambas cerraduras, estas deben ser idénticas, ya que la cerradura convexa es única.

3. Información extra

- Los resultados de sus comparaciones además de estar en un main (u otro archivo con fin de realizar el testing) deben venir en un archivo .pdf llamado **ANALISIS.pdf**, donde debe indicar

si tomó algún supuesto en las implementaciones y bajo qué condiciones obtuvo sus resultados, además de estos últimos.

- Se pide además graficar el tiempo obtenido para distintos valores de n como una forma de verifcar que su implementación sigue el comportamiento esperado, e incluir los gráficos en el archivo ANALISIS.pdf
- La fecha de entrega así como los contenidos de la tarea podrían llegar a modificarse según las necesidades del curso y cómo este avance, pero de ocurrir esto se notificará en clases y posteriormente se verá reflejado en U-Cursos.
- **Debe incluir README sobre cómo ejecutar su programa**.
- **Este tarea puede realizarse en grupos de máximo 2 integrantes**. De hacerlo así, se recomienda que cada integrante implemente un algoritmo cada uno, además de indicar en los comentarios de su entrega por u-cursos los nombres de ambos integrantes.